VQUANT

ALGORITHMIC TRADING DIVISION

PROJECT PROGRESS REPORT

Polymarket BTC Auto-Trading Bot

Phase 1: TradingView Pine Script v6 Signal Generator

PROJECT INFORMATION

CLIENT **Mr. Cam**	COMPONENT **Phase 1 - Pine Script**
LEAD DEVELOPER **Cuong**	DATE **April 23, 2026**
TARGET ASSET **BTCUSDT**	TIMEFRAME **Strictly 2-Minute (2m)**
OUTPUT FILE **polymarket_btc_bot_v2.pine**	PHASE 1 STATUS **100% Completed**

1 Phase 1 Progress Overview

Phase 1 focused entirely on building the "Brain" of the trading system. The team successfully programmed a custom indicator on TradingView using the latest **Pine Script v6** language. This indicator continuously analyzes BTCUSDT prices, applies complex mathematical filters, and automatically generates JSON Webhook payloads to trigger trades on Polymarket.

STATUS UPDATE

The entire Pine Script source code has been completed. Critical algorithms such as the Anti-repaint mechanism and the 10-minute window synchronization (Modulo 10) are functioning exactly as requested by the client (Mr. Cam).

1.1 Technical Solutions (Code Analysis)

To ensure the generated signals are highly precise and free from "Repainting" (false signals), we directly translated the client's explicit requirements into strict Pine Script logic. The system acts as a multi-layered mathematical filter:

1. ZERO-REPAINT TIMING CYCLE (MODULO 10)

The Polymarket target operates on a 5-minute cycle, but the script runs on a 2m chart for early execution. We enforce `minute % 10 == 0` to lock the signal strictly to the opening edges of :00, :10, :20, :30, :40, and :50. Anomalous candles (like :02 or :04) are completely bypassed. Coupled with `barstate.isconfirmed`, the webhook fires exactly once when the candle freezes.

2. VOLUME VECTOR SURGE

Calculates the Simple Moving Average (SMA) of the previous 10 candles' volume, explicitly excluding the current candle using the `[1]` operator to prevent look-ahead bias. The current candle's volume must exceed **200%** of this average (Multiplier = 2.0) to signify a genuine Smart Money surge.

3. ABSOLUTE BODY MOMENTUM FILTER

Filters out indecisive price action (like Dojis or Hammers). The script measures the absolute distance from Open to Close. To pass, this real body size must be **≥ 0.08%** of the closing price (`bodySize ≥ close * 0.0008`), ensuring only strong directional moves are traded.

4. MULTI-TIMEFRAME MACRO TREND & ADX

Cross-references the higher timeframe trend without leaking future data (using `lookahead = barmerge.lookahead_off`). A LONG signal strictly requires the price to be above both the 30m EMA (20) and 1h EMA (50). Furthermore, the **ADX (Average Directional Index) must be ≥ 20** to block sideways, choppy markets from triggering false breakouts.

5. PRODUCTION-READY GUARDRAILS

Incorporates a Duplicate Alert Guard (`var int lastAlertBar`) preventing multiple webhooks on the same candle, and an automated Timeframe Enforcer that warns users if applied to anything other than the strictly required 2m chart.

1.2 Webhook Payload (JSON Output)

When all 5 filters above pass simultaneously, the script fires a JSON payload to the Python backend. Example output:

```
{
  "symbol": "BTCUSDT",    "action": "LONG",
  "price":  77944.9,       "volume": 194,
  "time":   "2026-04-24T11:00:00+0000",
  "bar_index": 48231,  "adx": 28.4,
  "window_id":  "2026-04-24T11:00",
  "signal_id":  "BTCUSDT_20260424_110000",
  "source":     "polymarket_btc_bot"
}
```


2 Delivery & Verification

2.1 Validated Requirements & Proof of Execution

All core directives from the Technical Specifications have been fully realized. The visual proof of the system's operational readiness is detailed below.

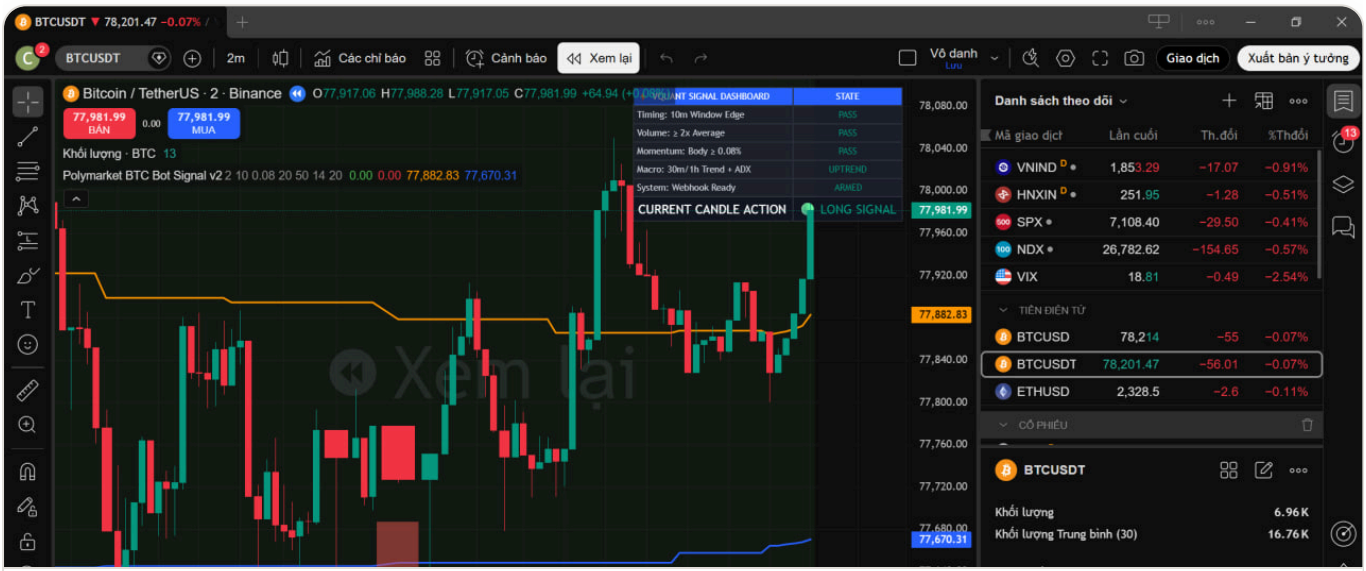

Fig 1. Real-time Forward Testing Dashboard running on BTCUSDT 2m.

CHECKLIST OF ACHIEVED CONDITIONS:

- ✓ **10-Minute Lock (Zero Repaint):** The bot rigidly waits for `barstate.isconfirmed` on the `% 10 == 0` candle, effectively bypassing all irrelevant candles (:02, :04, etc.) without a single false trigger during tests.

- ✓ **Volume Vector Confirmation:** Automatically calculates `SMA(10)` with a `[1]` lookback shift, completely filtering out weak breakouts by mandating a 200% volume surge.

- ✓ **Robust Absolute Body Sizing:** Tiny, insignificant Dojis are successfully rejected by strictly enforcing `bodySize ≥ close * 0.0008`, capturing only true momentum.

- ✓ **Advanced Trend Confluence (ADX & MTF):** Successfully pulls 30m/1h EMA lines securely via `lookahead_off` while enforcing ADX ≥ 20, actively blocking trades in sideways chop.

✓ **Enhanced JSON Webhook (v2 Upgrade):** Alerts are formatted with exact unformatted prices (`str.tostring(close)`) to preserve maximum decimal precision. Added crucial tracing parameters (`window_id`, `signal_id`, `bar_index`, `adx`) to enable flawless backend execution and logging.

✓ **Real-time Forward Testing Dashboard:** A premium on-chart UI was introduced to actively monitor all parameters (Timing, Volume, Momentum, Macro, System). It displays the exact "🟢 LONG SIGNAL" instantly upon all conditions aligning.

FORWARD-TESTING PLAN & INSTRUCTIONS

The source code is now fully prepared for live verification. We have embedded the **VQuant Signal Dashboard** directly into the indicator to make testing effortless.

How to test: Apply the script to a 2m chart. Observe the dashboard on the top right. It actively monitors all parameters in real-time. Wait for all conditions to align (Timing = PASS, Volume = PASS, etc.) to verify the exact moment the `🟢 LONG SIGNAL` or `🔴 SHORT SIGNAL` is fired. You can easily adjust the input parameters via the tooltips provided in the settings menu.

PHASE 1 READY FOR HANDOVER

Upon successful client confirmation of the forward-test, we will initiate Phase 2 (Python Webhook Backend & Polymarket API Integration).

3 Project Roadmap & Next Steps

With Phase 1 (Signal Generation) successfully delivered and verified, the project advances to the subsequent phases focusing on backend execution and cloud deployment.

3.1 Phase 2: Upstream Execution Server (Python Bot)

This phase builds the standalone backend server that receives signals from TradingView and executes trades on the Polymarket blockchain.

DECISION PIPELINE & RISK FIREWALL

Implements payload validation and a SHA-256 duplicate guard (60-second cache) to prevent accidental double-trades from redundant webhooks.

SMART MARGIN & FOK CEILING

Dynamic position sizing (fixed at 2.5% of total balance). Implements a strict Fill-Or-Kill (FOK) order system that drops any Ask price over $0.85.

3.2 Phase 3: Cloud Go-Live & Handover

The final phase ensures the bot is running 24/7 on a secure cloud environment, ready for automated real-money trading.

RAILWAY CLOUD DEPLOYMENT

Automated server build via Nixpacks on Railway. API keys and wallets secured via native environment variables.

SYSTEM HANDOVER

Delivery of source code, cURL sandbox testing scripts, and a detailed README manual for the client's operational management.

3.3 Overall Progress Timeline

DEVELOPMENT PHASE	CORE DELIVERABLES	STATUS
Phase 1: TradingView Pine Script	Signal Logic, Webhook Payload, Dashboard	**100% COMPLETED**
Phase 2: Python Execution Bot	Flask Server, Polymarket API, Risk Logic	**IN PROGRESS (NEXT)**
Phase 3: Deployment & Handover	Railway Setup, Testing, Documentation	**PENDING**